

Module 7 Week B — Integration Task:

Summarization & Integrated Evaluation Report

In this integration you will apply a pre-trained encoder-decoder summarization model to 120 tech / entertainment news articles, compute ROUGE (Recall-Oriented Understudy for Gisting Evaluation) against the reference summaries that ship with the dataset
(`data/tech_news_summaries/reference.csv` in your repo), and then write the **integrated evaluation report** — the synthesis that combines your Lab 7A fine-tuned classifier results, your Lab 7B QA (Question-Answering) results, and this task's summarization results into a single comparison report. **The integrated evaluation report is the Module 7 deliverable.**

Due: Thursday end of day. Resubmissions are accepted through Saturday of the assignment week.

What You Will Produce

Committed to your repo:

- `summarize.py`** — your implementation of the summarization pipeline.
- `Makefile`** — provided in the starter; you use `make summarize`, `make smoke`, `make clean`.
- Updated `README.md`** — 1–2 paragraphs documenting the model id, the tech news corpus version, and the `-n` run command.
- `summary_predictions.csv`** — 120 rows with reference summary, predicted summary, and per-summary ROUGE.
- `summary_metrics.json`** — aggregate ROUGE-1, ROUGE-2, ROUGE-L F1 with `n` and the model id.
- `integrated-evaluation-report.md`** — the six-section integrated report (the M7 deliverable).

The summarization model is loaded from Hugging Face Hub at runtime. **No model file** in your repo.

Setup

- Acquire the starter**

- Clone your repo:**

```
git clone <your-repo-url>
cd <your-repo-name>
```

- Install dependencies:**

```
pip install -r requirements.txt
```

This installs `transformers`, `torch` (CPU build), `pandas`, and `rouge-score`. The first three are already installed; `rouge-score` is the same package you installed for the drill (this is a separate repo, so a fresh `pip install` is needed).

- Create your working branch:**

```
git checkout -b integration-7b-summary-eval
```

First run: `python summarize.py` constructs a summarization pipeline, which downloads ~250 MB of model weights on first run. Plan ~3 minutes for the first run; subsequent runs use cached weights. The full evaluation on 120 articles completes in ~6–8 minutes on CPU after the model is cached.

What You're Building

`summarize.py` runs the full summarization evaluation end-to-end:

```
get_summarizer_model_name() # provided helper
build_summarizer(...)       # TODO (same as drill)
summarize_one(...)          # TODO (same as drill, with do_sample=False, num_s
compute_rouge(...)         # TODO (same as drill)
evaluate_summaries(...)     # TODO (new — orchestrates the harness)
main()                     # TODO (orchestration)
```

The starter `summarize.py` includes `get_summarizer_model_name()`, which returns `os.environ.get("SUMM_MODEL_FOR_CIT")` if set, otherwise `"ashleyfer/distilbart-cnn-6-6"`. Don't modify it — the autograder's smoke test depends on it.

Task 1: Pipeline + Single-Document Summarization

Implement `build_summarizer(model_name)` and `summarize_one(sum, text, max_length=150, min_length=30)` exactly as in the drill. Use `do_sample=False` and `num_beams=1` inside `summarize_one`.

These are the foundation. The autograder verifies them before checking the harness.

Task 2: ROUGE Scoring

Implement `compute_rouge(pred, ref)` exactly as in the drill — return `(f"rouge1": float, "rouge2": float, "rougeL": float)` with F1 measures, using a `RougeScorer` with `use_leserperd=True`. Remember: **the argument order**: `rouge_score(reference, predicted)`.

Task 3: Evaluate Over the Corpus

Implement `evaluate_summaries(summ, articles_df, refs_df)`:

- `articles_df` has columns `article_id`, `text` (the tech news articles).
- `refs_df` has columns `article_id`, `reference_summary`.
- Join on `article_id`, for each pair summarize the article and compute ROUGE against the reference.
- Return a dict:

```
{
    "rouge1": float, # mean ROUGE-1 F1
    "rouge2": float, # mean ROUGE-2 F1
    "rougeL": float, # mean ROUGE-L F1
    "n": int,
    "predictions": [
        {
            "article_id": str,
            "reference_summary": str,
            "predicted_summary": str,
            "rouge1": float,
            "rouge2": float,
            "rougeL": float,
            "frequency": float
        },
        ...
    ]
}
```

```
def evaluate_summaries(summ, articles_df, refs_df) -> dict:
    """
    Summarize each article and score against its reference.
    Returns aggregate ROUGE-1/2/L F1 plus per-article predictions.
    """
    # TODO: Join the two DataFrames on article_id
    # TODO: Iterate, summarize, compute ROUGE
    # TODO: aggregate and return
```

Task 4: Review `main()` (already wired up)

`main()` is already provided in `summarize.py` (lines 95–105). It uses the starter's private helpers (`articles_path()`, `references_path()`, `output_path()`) to read the `ARTICLES_PATH` / `REFERENCES_PATH` / `OUTPUT_PATH` environment variables (defaults: `data/tech_news_articles.csv`, `data/tech_news_summaries/reference.csv`, `summary_predictions.csv`). It then loads both CSVs, builds the summarizer via `build_summarizer(get_summarizer_model_name())`, calls `evaluate_summaries`, and writes:

- `summary_predictions.csv` (columns: `article_id`, `reference_summary`, `predicted_summary`, `rouge1`, `rouge2`, `rougeL`).
- `summary_metrics.json` (`rouge1`, `rouge2`, `rougeL`, `n`, `model`) — derived from the predictions output path (`csv` → `_metrics.json`).

You do not need to rewrite `main()` or add your own `os.environ.get` calls — the helpers handle path resolution. Open `main()`, read it, and confirm you understand the flow. Your remaining TODOs in Tasks 1–3 drive the actual computation.

Then run:

```
make summarize
```

This executes `python summarize.py` against the full 120-article evaluation set, ~6–8 minutes on CPU after the model weights are cached.

Task 5: Update the README

Edit `README.md` to add 1–2 paragraphs documenting:

- The summarization model name (default `ashleyfer/distilbart-cnn-6-6`) and a one-line description of what it is.
- The corpus version (120 tech news articles from M8) and the reference summaries file.
- The re-run command (`make summarize`).

This is the reproducibility-minimum requirement — anyone reading your repo should be able to re-run your evaluation.

Task 6: The Integrated Evaluation Report — the M7 Deliverable

This is the central artifact. The `integrated-evaluation-report.md` synthesizes three weeks of measurement:

- Lab 7A** — your fine-tuned DistilBERT classifier on AARSynth app reviews. Use the macro-F1 from `your_metrics.json`.
- Integration 7A** — the same fine-tuned classifier applied to the tech news corpus (domain shift; included for engineering context).
- Lab 7B** — pre-trained QA on the curated tech-news QA set. Use EM and token-F1 from your `qa_metrics.json`.
- Integration 7B (this task)** — pre-trained summarization on the tech news corpus. Use ROUGE from your `summary_metrics.json`.

Manually paste the relevant numbers from each metrics file into the report's comparison table. **The TA verifies that the report's numbers match your submitted metrics files.**

The report has **six required sections**. The starter includes a template `integrated-evaluation-report.md` with these section headers — fill them in.

Section 1: Comparison Table

Task	Approach	Model	Training cost	Inference cost	Quality metric	Value
Sentiment classification (Lab 7A)	Fine-tuning	DistilBERT	~30 min CPU + 3K labels	~50 ms / example	Macro-F1	(from your <code>metrics.json</code>)
Domain transfer of fine-tuned classifier (Integration 7A)	Fine-tuned model on out-of-domain	(same)	(already trained)	~50 ms / example	Domain-shift judgment	(qualitative — from Integration 7A)
Extractive QA (Lab 7B)	Pre-trained inference	distilbert-base-cased-distilled-squad	0	~50 ms / example	EM / token-F1	(from your <code>qa_metrics.json</code>)
Abstractive summarization (Integration 7B)	Pre-trained inference	distilbert-cnn-6-6	0	~3 sec / example	ROUGE-1/2/L F1	(from your <code>summary_metrics.json</code>)

Section 2: Findings

3–5 bullet points characterizing what each approach excels at and where it breaks. Examples of the kind of finding to write:

- "Fine-tuning produced a 0.87 macro-F1 on app-review sentiment but suffered visible domain-shift breakdown on tech news articles (Integration 7A)."
- "Pre-trained QA achieved [your EM] / [your F1] on the curated tech-news QA set; your failure-mode insight from Lab 7B).

Make the bullets specific to your numbers.

Section 3: Faithfulness Check (qualitative)

Pick **three summaries** from `summary_predictions.csv` (one high-ROUGE, one mid-ROUGE, one low-ROUGE). For each:

- Quote the article and the predicted summary.
- Mark whether the summary is faithful (every claim appears in the article).
- Comment on what ROUGE caught or missed for this summary.

This is the section that puts the limits of ROUGE on the page concretely. A high-ROUGE summary that hallucinates is the canonical example to call out — if you find one, this section is gold.

Section 4: Production Decision Matrix

For each of three hypothetical production scenarios, recommend either fine-tuning or pre-trained inference, with one **specific sentence justifying** the recommendation grounded in your measured numbers.

Scenario	Recommendation	Justification (1 sentence, tied to your numbers)
Real-time app-review sentiment dashboard for trading desk	(your call)	(your justification)
Internal tech / entertainment news summary digest for a newsroom team	(your call)	(your justification)
Domain-expert QA on legal contracts	(your call)	(your justification)

A weak version of this section says "it depends" everywhere. A strong version specifies what *makes it depend* — labeled-data availability, latency budget, faithfulness requirement.

Section 5: What You Would Do Differently

One paragraph on what you would change about your approach if you had a labeled summarization dataset for the tech / entertainment news domain. Concrete: would you fine-tune a summarizer? Train a calibration model on top? Invest in a faithfulness audit? The goal is to articulate one engineering investment that would meaningfully change the numbers.

Section 6: Limits of the Evaluation

One paragraph on what these numbers do not tell you. Examples of what to address:

- ROUGE doesn't capture faithfulness — your faithfulness check (Section 3) addresses some of this, but only on three summaries.
- EM / token-F1 don't capture calibration — a model can be 80% accurate and 100% confident on every output.
- Latency under load isn't measured — these are single-request numbers.

Don't list every possible limit. Pick the one or two that matter most for the production scenarios you discussed in Section 4.

Summitting

Everything is done:

- Stage and commit:**

```
git add summarize.py Makefile README.md summary_metrics.json summary_predictions.csv
git commit -m "Complete integration 7b"
```

- Push to the remote repository.**

- Open the Pull Request** from your working branch into `main`. The autograder runs `make smoke` against a tiny fixture (3 articles + same reference file) using a substitute model — this verifies your pipeline is structurally correct end-to-end. It does not re-run your full 120-article evaluation.

Your PR description must include:

- Aggregate ROUGE-1, ROUGE-2, ROUGE-L F1 (two decimals each).
- One-sentence summary of your production decision matrix's central recommendation (e.g., "Pre-trained inference is sufficient for low-stakes summarization; fine-tuning is justified for the trading desk classifier where labeled data exists").
- Paste your PR URL into `Talent.MS` → Module 7 → Integration 7B to submit this assignment.

Challenge Extensions

The following extensions are optional; they extend the integration but do not replace any required deliverable. They are graded as part of the rubric portion when the base assignment is complete.

Challenge Tier 1 — Cross-modal: speech-to-text with Whisper

This tier carries you from "the encoder-decoder family handles text-to-text generation" (your summarization pipeline) to "the same architectural family handles speech-to-text" (Whisper). The pedagogical payoff is generalization: you will see that the paradigm — encoder consumes input, decoder generates output token-by-token via cross-attention — is what matters, not the modality.

Concept primer. Whisper (a family of encoder-decoder transformer models from OpenAI) for **automatic speech recognition (ASR)** — converting audio waveforms into text transcripts. The encoder reads a mel-spectrogram of the audio (frequency content over time, treated as a sequence of feature vectors) and produces hidden states. The decoder generates the transcript autoregressively, attending to the encoder's hidden states via cross-attention — structurally identical to how BART's decoder generates summary tokens while attending to the encoder's article representation. The only thing that changes is the input encoder's expectations (audio features instead of text tokens) and the training corpus (audio-transcript pairs instead of article-summary pairs).

This tier uses `openai/whisper-tiny.en` (~75 MB, ~30M params, English-only checkpoint — the LibriSpeech dummy data you'll evaluate against is English, so the `_en` variant is appropriate; the multilingual checkpoint `openai/whisper-large` is available if you want to experiment beyond this tier).

Whisper comes in sizes — `tiny`, `base`, `small`, `medium`, `large` — that trade quality for compute. `tiny` is CPU-feasible at ~3–10 seconds per 10-second audio clip; the larger variants get steadily better at noisy audio, accents, and rare vocabulary, but require GPUs at usable latency. You evaluate ASR with **word error rate (WER)**: $(\text{substitutions} + \text{insertions} + \text{deletions}) / \text{total_reference_words}$, computed via the `jiwer` library (`pip install jiwer`). WER is a cost metric — lower is better. 0.0 is perfect — which is the opposite convention from accuracy. F1, ROUGE, the two production recommendations name specific models with thresholds grounded in human review before publication; this would route NLP of summaries to human review based on the 30-summary audit.

WER normalization (do this explicitly). `jiwer.wer()` by default applies a minimal transform (whitespace cleanup + tokenization). It does not lowercase, strip punctuation, or expand contractions unless you tell it to. For an honest WER comparison against Whisper's punctuated, cased output, build an explicit pre-processing pipeline using `jiwer.Compose` ([JiWER transformations docs](#)):

```
import jiwer

normalize = jiwer.Compose([
    jiwer.ToLowerCase(),
    jiwer.RemovePunctuation(),
    jiwer.RemoveMultipleSpaces(),
    jiwer.Strip(),
    jiwer.ReduceToListOfListOfWords(),
])
```

```
wer_value = jiwer.wer(normalize(reference_text), normalize(predicted_text),
                      truth_transform=normalize,
                      hypothesis_transform=normalize)
```

Document the transform you used in the report — different normalization choices produce different WER numbers and "WER = 0.12" without a normalization spec is not a reportable result.

Task.

- `pip install jiwer` in your integration venv (no other new dependencies — Whisper loads via your existing `transformers` install).

- Write a committed, runnable script `asr_eval.py` at the repo root with a `--init` CLI flag. Running `python asr_eval.py --init 50` must produce both `asr_predictions.csv` and `asr_metrics.json` from scratch. Skeleton:

```
# asr_eval.py
import argparse
from datasets import load_dataset
from transformers import pipeline

MODEL_ID = "openai/whisper-tiny.en"

NORMALIZE = jiwer.Compose([
    jiwer.ToLowerCase(), jiwer.RemovePunctuation(),
    jiwer.RemoveMultipleSpaces(), jiwer.Strip(),
    jiwer.ReduceToListOfListOfWords(),
])

def main(limit: int) -> None:
    ds = load_dataset("ljspeech", split="train", use_auth_token="")
    clips = ds.select(range(limit))
    asr = pipeline("automatic-speech-recognition", model=MODEL_ID)
    rows = []
    for i, row in enumerate(ds):
        out = asr(row["audio"])["text"]
        predicted = out
        reference = row["text"]
        wer = jiwer.wer(reference, predicted,
                        truth_transform=NORMALIZE,
                        hypothesis_transform=NORMALIZE)
        rows.append({"audio_id": f"clip_{i:03d}",
                    "reference": reference, "predicted": predicted,
                    "wer": round(wer, 3)})
    # write asr_predictions.csv (4 columns) and asr_metrics.json
    # ({"wer": aggregate, "n": count, "model": MODEL_ID,
    #  "normalization": jiwer.Compose(...).transformer})
    ...
```

If `--name` == `"main"`, `p = argparse.ArgumentParser()` `p.add_argument("--init", type=int, default=50)` `main(p.parse_args()).limit()`

The TA will run `python asr_eval.py --init 50` to verify your committed CSV/JSON match what the script produces.

- Correctly aggregate WER across all clips using `jiwer.list_of_refs`, `list_of_preds`, `truth_transform=...`, `hypothesis_transform=...` — passing lists computes a corpus-level WER (comparatively weighted by reference length), which differs from the mean of per-clip WERs. Use corpus WER for the headline number in `asr_metrics.json`.

- "What the model heard wrong" mini-analysis.** Pick 3 clips from your run (one with low WER, one mid, one high — sort `asr_predictions.csv` by `wer` to find them). For each, identify one human-interpretable error category:

- misheard word (deletion)
- substitution (heard a different word than what was said — often a phonetic neighbor)
- insertion (added a word that wasn't said)
- punctuation/casing artifact (Whisper added punctuation the reference doesn't have; not a true transcription error but it inflates WER without `RemovePunctuation`)
- name-face failure (model couldn't transcribe a proper noun)

Record the `audio_id`, reference, predicted, category, one-sentence diagnosis) for each of the 3 in a small table made via your report subsection (see step 5).

- Add Section 7 — Challenge Extensions** to your `integrated-evaluation-report.md` if it doesn't already exist (this is the umbrella section for any challenge tier work). Inside it, add a subsection:

Section 7.1 — Cross-Modal Observation. One paragraph + a one-row addition to the Section 1 comparison table + the 3-clip "what the model heard wrong" table from step 4. The paragraph addresses: how does Whisper-tiny's quality/latency trade-off (tiny vs. base vs. small) parallel the fine-tuning-vs-pre-trained-inference decision you made for QA and summarization? When would you reach for Whisper-tiny in production vs. a paid speech-to-text API?

Time budget for a strong learner: ~2–5 hours. Rough breakdown: 30–45 min writing `asr_eval.py` cleanly with the `jiwer.Compose` normalization transform; 5–10 min running it and sanity-checking the CSV, inspecting WER, 20–30 min on the 3-clip "what the model heard wrong" mini-analysis (sorting the CSV, inspecting reference vs. predicted, categorizing). 45–60 min writing Section 7.1 thoughtfully (paragraph + comparison table row + 3-clip table). This is slightly above the standard Tier 1 normal of 1.5–2 hrs because the committed-script requirement + structured 3-clip analysis is heavier than a typical Tier 1.

Optional path — bring your own clip (advanced; skip unless trivially possible). If you already know how to load a WAV file into a 16 kHz `float32` numpy array without new system dependencies (no `librosa`, no `ffmpeg`, no new pip installs), record or find a 5–10s clip and run Whisper-tiny on it. Add the transcript to your "what the model heard wrong" table. Otherwise skip — your audio loading is out of scope for this tier and not worth the install friction. The LibriSpeech-dummy path is the supported route.

What earns Tier 1 credit: `asr_eval.py` is committed and runs end-to-end with `python asr_eval.py --init 50` producing both deliverables: the `jiwer.Compose` normalization transform is explicit in code and named in `asr_metrics.json`; the comparison-table row uses the corpus WER from your committed JSON (not your estimate); the 3-clip "what the model heard wrong" table identifies a real, named error category per clip; and the paragraph explains one quantitative condition (e.g., "Whisper-tiny is appropriate when WER < X on a representative sample of the production audio distribution AND latency budget tolerates ~5s per 10s clip") rather than "it depends."

Challenge Tier 2 — Faithfulness audit at scale

The base assignment's Section 3 asks you to inspect three summaries for faithfulness — that's a sample size of three, which is anecdote, not measurement. This tier turns the qualitative spot-check into a counted audit you can defend to a stakeholder.

Concept primer. Faithfulness in summarization means: the summary's factual claims are entailed by the source article. A summary can have high ROUGE (lots of n-gram overlap) and still be unfaithful — it can name the wrong actor, hallucinate a number, or contradict an attribution while reusing surface-level vocabulary from the article. ROUGE doesn't catch this; humans (or, increasingly, model-based faithfulness scorers) do. For the pilot cohort scale, a structured human audit is the practical instrument.

Task.

- Sample 30 summaries from your `summary_predictions.csv` (random 30; document your seed for reproducibility).

- Apply the 4-category taxonomy. For each summary, tag with zero, one, or more of:

- HE — Hallucinated entity.** The summary names a person, organization, place, or product not in the article.
- HN — Hallucinated numeric.** The summary cites a value, percentage, date, or count that is not in the article — or that contradicts a number that is.
- DE — Dropped key entity.** The article's lede sentence names a primary actor that the summary omits entirely.
- CA — Contradicted attribution.** The summary attributes a statement or action to the wrong actor relative to the article.

- Produce `faithfulness_audit.csv` with columns: `article_id`, `predicted_summary`, `tags`, `notes`. The `tags` column is a pipe-delimited list (e.g., `HE|DE` or empty for faithful summaries). The `notes` column is a 1–2 sentence justification for each tag.

- Rewrite Section 3 of your `integrated-evaluation-report.md`** — it should be structured by counts rather than anecdote. The new Section 3 must include: (a) total counts and percentages for each of the four tags across the 30-summary audit, (b) one illustrative example per tag (cite `article_id`, the offending span of the predicted summary, and the article evidence), (c) a production rule grounded in your measured frequency — e.g., "summaries flagged HE OR CA require human review before publication; this would route NLP of summaries to human review based on the 30-summary audit."

Time budget for a strong learner: ~3–5 hours. Most of the time is reading the original article alongside the predicted summary thoughtfully — entry-level faithfulness is genuinely time-consuming. Expect the taxonomy to feel slightly off on a few cases; document edge cases in your `notes` column rather than forcing them into the wrong category.

What earns Tier 2 credit: counts are real (the CSV's tag distribution matches the reports' numbers); the production rule's routing percentage is computed from your measured frequency, not made up; at least three of the four taxonomy categories have a non-trivial example in the audit (not "all 30 were faithful").

Challenge Tier 3 — Summarizer Pareto frontier

The base assignment picks `ashleyfer/distilbart-cnn-6-6` because it's the size/quality compromise the build team chose. This tier turns the choice into your own measured Pareto frontier: which summarizer would you select for two specific production scenarios, and what numbers defend your choice?

Concept primer. Pareto-optimal means: no other model in the candidate set is better on every axis you care about. If model A has higher ROUGE-L but model B is faster at every latency budget that matters, both are Pareto-optimal — you pick between them based on which axis the scenario weights heavier. A model that is worse than another on every axis is Pareto-dominated and should be eliminated from consideration. This is the most honest decision frame for production model selection: you don't pick "the best model," you pick "the best model for this scenario's weighting," and you make the weighting explicit.

Task.

- Refactor `summarize.py`** so the model id is a parameter, not a hardcoded constant. Add a `--model` CLI flag (or read from an environment variable). The same harness must work for any HF summarization pipeline-compatible model id.

- Run the harness against three additional summarizers on the same 120 tech-news articles:

- `tf-small` (~250 MB) — uses a "summarize" task prefix; small encoder-decoder.
- `ashleyfer/distilbart-xsum-12-1` (~400 MB) — distilled BART trained on the XSum corpus (highly compressed single-sentence reference style).
- `ashleyfer/distilbart-cnn-12-6` (~470 MB) — larger sibling of your baseline distilBART, same CNN4M training corpus.

- For each of the four models (your baseline + three new), record into `model_comparison.csv`:

- `model` — HF id
- `rouge1`, `rouge2`, `rougeL` — aggregate F1 across the 120 articles
- `mean_latency_sec` — mean per-article inference wall time (use `time.time()` around the `summarize(...)` call; warm-up with one unfixed call first to amortize model-load)
- `disk_mb` — model footprint in disk (look at `model.config.to_dict()["tokenizer_config"]["vocab_size_embeddings"]` after the run)

- Produce a Pareto-frontier plot (matplotlib).** Scatter id, ROUGE-L F1 (y-axis) vs. mean inference latency (x-axis); annotate each point with its model id; draw a line connecting the Pareto-optimal models. Save as `model_pareto.png`.

- Add Section 7 — Challenge Extensions** to your `integrated-evaluation-report.md` if it doesn't already exist (this is the umbrella section for any challenge tier work). Inside it, add a subsection:

Section 7.2 — Multi-Model Production Selection. The subsection must:

- Embed `model_pareto.png` (or describe it precisely in prose if image embeds are awkward).
- Recommend a Pareto-optimal model for two production scenarios: (a) **batch backend** — latency tolerant (overnight runs where fine, quality maximized); (b) **real-time API** — latency budget ≤ 1 s per article, quality acceptable above a stated threshold.
- State the quality threshold below which you reject a model regardless of latency advantage (e.g., "ROUGE-L < 0.20 is unusable for the trading-desk summary feed because end-users complained of unintelligible summaries at that quality in our hypothetical pilot").
- Identify any Pareto-dominated model in your candidate set and recommend dropping it from future consideration.

Time budget for a senior engineer (5+ years professional experience): ~5–8 hours. Most of the time is the refactor (the per-model generation kwards vary — T5 needs the "summarize:" prefix; the distilBART models share kwards but you need to defensively confirm), the four 120-article runs (~20 min compute total but can run sequentially), the latency instrumentation done correctly (warm-up handling, repeat measurement, mean vs. median), and the Pareto reasoning done with discipline (no "it depends" — name the scenarios and the thresholds).

Prerequisite. Base assignment substantially complete — you need the distilBART baseline numbers to anchor the comparison.

What earns Tier 3 credit: `summarize.py` is genuinely model-agnostic (a TA can run it against a 5th model by passing one CLI flag, with no code change); per-model inference numbers are measured with warm-up handling, not just first-call timings; the Pareto plot correctly identifies the dominated model(s); the two production recommendations name specific models with thresholds grounded in your measured ROUGE-L and latency numbers, not "depends on the use case."

What's Next

Tomorrow (Thursday) — Honors Track only: Stretch — summarize-then-QA two-step pipeline (extends this integration on branch `stretch-thu-summarize-thu-q1`).

